1. Target Group

In AWS, a **Target Group** is a logical group of backend resources that receive traffic from a load balancer.

In our 3-tier architecture, the targets are **EC2 instances**.

So, conceptually:

```
Load Balancer → Target Group → EC2 Instances
```

The **target group acts as the bridge between the load balancer and the servers**.

2. Does the Frontend Load Balancer Have Any Business with Target Groups?

Yes - **a load balancer cannot work without a target group**.

The **target group is what tells the load balancer where to send traffic**.

For example:

```
User Request
     ↓
Frontend ALB
     ↓
Target Group (Web Tier)
     ↓
Frontend EC2 Servers
```

So the load balancer:

1. receives a request
2. checks its **listener rules**
3. forwards the request to a **target group**

3. Example From Our Architecture

Our system has **2 load balancers**.

Frontend ALB

Receives traffic from the internet.

Target group example:

```
citymart-frontend-alb-tg
```

Targets inside it:

```
EC2 Web Server 1
EC2 Web Server 2
```

Traffic flow:

```
Internet
   ↓
Frontend ALB
   ↓
Web Target Group
   ↓
Web EC2 Instances
```

Backend ALB (Internal)

This load balancer distributes traffic to the **application servers**.

Target group example:

```
citymart-backend-alb-tg
```

Targets inside it:

```
App Server 1
App Server 2
```

Traffic flow:

```
Web Servers
     ↓
Internal ALB
     ↓
App Target Group
     ↓
Backend EC2 Instances
```

4. What a Target Group Actually Does

A target group performs several important functions.

1. Load Balancing

The ALB distributes requests across instances in the target group.

Example:

```
Request 1 → Server A
Request 2 → Server B
Request 3 → Server A
```

Common algorithms:

- Round robin
- Least outstanding requests

2. Health Checks

Target groups perform **health checks** to determine whether instances are healthy.

Example health check:

```
HTTP:80 /health
```

If an instance fails health checks:

```
Load Balancer stops sending traffic to it
```

Example:

```
Server A → healthy
Server B → unhealthy

ALB sends traffic only to Server A
```

3. Automatic Registration

When new instances are launched (via Auto Scaling), they are **automatically registered into the target group**.

5. Auto Scaling Group

An **Auto Scaling Group (ASG)** manages a group of EC2 instances.

It automatically:

- launches instances
- terminates instances
- replaces unhealthy instances
- scales capacity based on demand

Our configuration:

```
Minimum: 2 instances
Desired: 2 instances
Maximum: 2 instances
```

6. Relationship Between ASG and Target Group

This is extremely important.

The **Auto Scaling Group attaches to the Target Group**.

Example:

```
Auto Scaling Group
      ↓
Registers instances to
      ↓
Target Group
      ↓
Used by
      ↓
Load Balancer
```

So when ASG launches a new instance:

```
New EC2 Instance
      ↓
Automatically added to Target Group
      ↓
Load Balancer starts sending traffic
```

And if an instance is terminated:

```
Removed from Target Group
```

This makes the system **self-healing and scalable**.

7. Example in Our Architecture

Web Tier

```
Frontend ALB
     ↓
Web Target Group
     ↓
Auto Scaling Group
     ↓
Web EC2 Instances
```

Example:

```
Min: 2
Desired: 2
Max: 2
```

Application Tier

```
Internal ALB
      ↓
App Target Group
      ↓
Auto Scaling Group
      ↓
App EC2 Instances
```

8. Full Request Flow in Our System

Putting everything together:

```
User Browser
      ↓
Route53 DNS
      ↓
Frontend ALB
      ↓
Web Target Group
      ↓
Web EC2 Instances (ASG)
      ↓
Internal ALB
      ↓
App Target Group
      ↓
App EC2 Instances (ASG)
```

```
      ↓
Aurora MySQL Database
```

9. Why This Architecture Is Powerful

This design gives us:

High Availability

Instances run in **multiple availability zones**.

Fault Tolerance

If a server fails:

```
ASG replaces it
```

Elastic Scaling

If traffic increases:

```
ASG launches more instances
```

Load Distribution

Traffic is spread across servers.

10. Simple Way to Remember

Think of it like this:

```
Load Balancer = Traffic Controller
Target Group = List of Servers
Auto Scaling Group = Server Factory
```

So, the system works like:

```
Traffic Controller
      ↓
Server List
      ↓
Server Factory creates/removes servers
```

cba_admin

EC2 > Target groups

Snapshots
Lifecycle Manager

▼ **Network & Security**

Security Groups
Elastic IPs
Placement Groups

Target groups (2) Info | What's new?

Actions ▼ | Create target group

Filter target groups

1

Name	ARN	Port	Protocol	Target type	Lo
citymart-backend-alb-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	cit
citymart-frontend-alb-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	cit

cba_admin

EC2 > Target groups

- Volumes
- Snapshots
- Lifecycle Manager
- **Network & Security**
  - Security Groups
  - Elastic IPs
  - Placement Groups
  - Key Pairs
  - Network Interfaces
- **Load Balancing**
  - Load Balancers
  - Target Groups
  - Trust Stores
- **Auto Scaling**
  - Auto Scaling Groups

Target groups (1/2) Info | What's new?

Actions ▼ | Create target group

Filter target groups

1

Name	ARN	Port	Protocol	Target type	Lo
citymart-backend-alb-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	cit
citymart-frontend-alb-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	cit

Target group: citymart-frontend-alb-tg

Instance ID	Name	Port	Zone	Health status	Health status detai
i-0ba42e16a06c86f40	Citymart-web-...	80	eu-west-2b (eu...	Healthy	-
i-04061f4608f29dcd9	Citymart-web-...	80	eu-west-2a (eu...	Healthy	-

EC2 > Target groups

Volumes
Snapshots
Lifecycle Manager

Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces

Load Balancing
Load Balancers
Target Groups
Trust Stores

Auto Scaling
Auto Scaling Groups

Target groups (1/2) Info | What's new?

Actions ▼ | Create target group

Filter target groups

Name	ARN	Port	Protocol	Target type	Lo
citymart-backend-alb-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	cit
citymart-frontend-alb-tg	arn:aws:elasticloadbalancin...	80	HTTP	Instance	cit

Target group: citymart-backend-alb-tg

Instance ID	Name	Port	Zone	Health status	Health status det
i-09f1656d474355f2e	Citymart-app-t...	80	eu-west-2b (eu...	Healthy	-
i-026197610a2ddafda	Citymart-app-t...	80	eu-west-2a (eu...	Healthy	-